

Promptimo — Résumé de l'application

SaaS de gestion et de prospection pour agents immobiliers indépendants

1. Contexte et problématique

Les agents immobiliers indépendants passent en moyenne 30 à 45 minutes à rédiger chaque annonce manuellement, gèrent leurs contacts dans des outils non adaptés (Excel, carnet papier) et jonglent entre plusieurs applications sans vue d'ensemble sur leur portefeuille. Promptimo répond à cette problématique en centralisant la gestion de l'activité d'un agent (clients, biens, annonces, carte) au sein d'une seule plateforme web, et en intégrant la génération automatique d'annonces et d'emails professionnels grâce à l'intelligence artificielle.

2. Fonctionnalités principales

L'application couvre quatre modules fonctionnels :

- Authentification sans mot de passe : connexion par lien magique envoyé par email (magic link), session JWT chiffrée (JWE). Inscription avec validation du SIRET (algorithme de Luhn + API INSEE), paiement par abonnement via Stripe Checkout, activation du compte par magic link signé.
- Gestion des clients et des biens : CRUD complet avec filtres, recherche, association client ↔ bien, géocodage automatique des adresses via l'API Nominatim (OpenStreetMap).
- Génération d'annonces IA : à partir des caractéristiques d'un bien, un prompt structuré est envoyé au modèle Anthropic claude-sonnet-4-6. La réponse est retransmise en temps réel par streaming SSE, affichée avec un curseur clignotant, puis sauvegardée en base. L'annonce peut être régénérée à la demande.
- Génération d'emails IA : création d'emails professionnels contextualisés (suivi de visite, relance prospect, présentation de bien, offre reçue) en sélectionnant un type, un client et un contexte libre.
- Carte interactive : visualisation des biens géolocalisés sur une carte Leaflet (OpenStreetMap), marqueurs colorés par type de transaction, popups avec détails, repositionnement par glisser-déposer avec sauvegarde des coordonnées.

3. Stack technique

Promptimo est une application full-stack Next.js 16 (App Router), développée en TypeScript strict. L'ensemble du frontend et des routes API est contenu dans un seul projet — il n'y a pas de backend séparé.

- Frontend : React 19, Tailwind CSS v4, shadcn/ui (composants accessibles base-ui), react-leaflet pour la carte, react-hook-form + Zod pour les formulaires.
- Backend (API Routes) : Next.js App Router, validation Zod à chaque entrée, sessions JWT/JWE via NextAuth v5.
- Base de données : PostgreSQL 18 via Prisma v7 (ORM, migrations versionnées, adaptateur pg natif). Modèle en 6 entités : Agency, Agent, Client, Property, VerificationToken, PendingRegistration.
- Services tiers : Anthropic SDK (génération IA en streaming), Stripe SDK v22 (abonnements + webhooks), Resend (emails transactionnels), Nominatim (géocodage).
- Infrastructure : VPS Ionos S+ (Ubuntu 26.04), PM2 (process manager), Nginx (reverse proxy), Node.js 24 (npm).

4. Sécurité

Plusieurs mécanismes de sécurité sont implémentés à différents niveaux :

- Authentification passwordless : aucun mot de passe stocké, token SHA-256 à usage unique, session JWE chiffrée avec AUTH_SECRET, middleware Next.js sur toutes les routes protégées.
- Isolation multi-tenant : chaque requête API vérifie session.user.id (agentId). Toutes les requêtes Prisma incluent un filtre WHERE { agentId } — un agent ne peut accéder qu'à ses propres données.
- Validation des entrées : schémas Zod sur chaque route API, validation SIRET en deux étapes (Luhn + API officielle), vérification de signature HMAC-SHA256 sur les webhooks Stripe.
- Secrets : fichiers .env exclus du dépôt Git dès le premier commit, secrets CI/CD dans GitHub Actions Secrets, .env de production créé directement sur le VPS.

5. Tests et déploiement

La stratégie de tests couvre trois niveaux : 7 tests unitaires Vitest (buildListingPrompt), 18 tests d'intégration Vitest sur les API routes avec BDD PostgreSQL réelle (tunnel SSH), et 4 tests End-to-End Playwright (parcours complets sur Chromium). Les 29 tests s'exécutent automatiquement à chaque push via un pipeline GitHub Actions. Le déploiement sur le VPS est conditionnel à la réussite complète des tests (SSH + git pull + npm build + pm2 restart). La production n'est jamais mise à jour si un test échoue.